535 Project Final Report

James Dolle | Pablo Ghezzi

ARM LEGv8 Architecture

General Description:

Project statement:

Our Instruction Set Architecture will be a general-purpose architecture heavily based on ARM LEGv8. ARM LEGv8 is a smaller subset of instructions that uses 32-bit instructions with 6 key instruction formats. Our group will use the ARM LEGv8 green card for ISA reference. Additionally, we will only implement signed integer-based operations and will implement a graphical operation unit.

Project Layout:

Our implementation will use a 32-bit architecture with 16 registers and 32-bit word size. We will implement most of the ARM LEGv8 instructions with a priority on the basics. This includes Boolean operations, ADD, SUB, SHIFT, BRANCH, COND BRANCH, LOAD/STORE.

The ISA simulations 16 registers will contain both specific and general purpose registers. This will include features such as the 0 immediate, FLAG, and return address register. The ARM LEGv8 uses 32 registers but for our simplified ISA simulation, 16 registers will provide ample capacity. ARM LEGv8 uses 8 separate instruction formats we will implement to include all required instructions for the project. Below is an example instruction format used for ADD and SHIFT operations that we will implement.

	Byte 0								**Byte 1**								**Byte 2**								**Byte 3**							
Instruction type	**31**	**30**	**29**	**28**	**27**	**26**	**25**	**24**	**23**	**22**	**21**	**20**	**19**	**18**	**17**	**16**	**15**	**14**	**13**	**12**	**11**	**10**	**9**	**8**	**7**	**6**	**5**	**4**	**3**	2	1	0
R	Type Field			Opcodes					0				Rm				0			shamt					Rn				Rd			

Instruction fields	Field description
Type Field	Instruction type
Opcode	Operation
Rm	Second source register
Shamt	Shift amount, used for shift instructions but ignored for ADD instructions.
Rn	First Source register
Rd	Destination register

Each word fetched from memory will contain a single instruction for processing. The memory will be a Princeton design that will have a byte addressing scheme that allows for both byte and word size LOAD/STORE instructions. The memory will have 16 bit addressing for a max memory size of 64KB. The cache will initially be a one-layer cache but will possibly be improved to be a two-layer cache later in the course. The cache will be directly mapped with a 1KB size with 8 word line length. Further details of cache bit layout described in *Memory/Cache System*.

Register Specifics:

For our registers, we will have a total of 16 registers, each with a size of 32 bits.

NAME	NUMBER	USE
X0 - X12	0 - 12	General Purpose
X13 (SP)	13	Stack Pointer
X14 (PC)	14	Program Counter
XZR	15	Zero Register
FLAGS	NA	Negative | oVerflow | Zero | Carry

Flag Bits: NZVC

- **N** - Negative: Flag when the result of an operation is negative, 4'b1000
- **Z** - Zero: Flag when the result of an operation is zero, 4'b0100
- **V** - Overflow: Flag when the result of operation overflows, 4'b0010
- **C** - Carry: Flag for when operating carries a bit, 4'b0001

Included Instructions:

For the instructions, we will have all the basic arithmetic operations such as Addition, subtraction, Multiplication, and Division. We will also have Logical operations such as Bitshifts, Bitwise OR, and AND. Most operations have associated flags that they will set as listed in the opcode section. Load-store operations that interact with the memory will allow you to load/store bytes and words.

We will have branching, both conditional and unconditional. For conditional branching, subtract to set the flags, then use a conditional branching instruction. The conditional branch will check the set flags depending on the condition. Example to check if A < B then instruct A - B and check the < condition. Another example, for A >= B do A - B and check the >= condition. Each condition is tied to what flags are set.

- Equal (==), Z==1,
- Not equal (!=), Z==0
- Greater than (>), N==V && Z==0
- Greater than or equal (>=), N==V
- Less than (<), N!=V
- Less than or equal (<=), N!=V || Z==1

Special Features:

<u>ISA Modes</u>:

Our ISA Architecture will allow for four operation modes as per the guidelines. The mode can be configurable in the GUI for ease. The four modes are (1) disabling the pipe and cache, (2) memory-only operation, (3) pipe-only operation, and (4) cache and pipe function normally.

These modes will allow our group to hone in on specific components of our ISA simulation to assist in troubleshooting any errors we may have. For instance, to ensure the pipeline is functioning properly we may want to enable the pipe-only operation.

<u>PPU</u>:

The CPU will be connected to a PPU (picture processing unit), a component inspired by the unit of the same name included inside of the NES. The PPU will be controlled via memory locations that send data to the PPU. It will output to 256 x 224 pixels.

See **Appendix C** for a rough diagram of the PPU functioning.

Instruction Details:

All instructions are a constant size of one word (32 bits) and are in one of these 8 instruction formats.

	Byte 0								**Byte 1**								**Byte 2**								**Byte 3**							
bit	**31**	**30**	**29**	**28**	**27**	**26**	**25**	**24**	**23**	**22**	**21**	**20**	**19**	**18**	**17**	**16**	**15**	**14**	**13**	**12**	**11**	**10**	**9**	**8**	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**
H	0																															
R	001			Opcodes					0				Rm				0			shamt					Rn				Rd			
I	010			Opcodes					ALU Immediate																Rn				Rd			
D	010			Opcodes					DT Address																Rn				Rd			
B	100			Opcodes					Branch Address																							
IW	101			Opcodes					Move Immediate																0		SH		Rt			
ME	110			Opcodes					IMM / Rt																							
NOP	111			EMPTY																												

Name
mne(opcode): **Mnemonic** (Opcode (Hex))
Format: Format
Flags Set: Sets Flags
Operation: Operation

HaLT
mne(opcode): **HLT** (Any non-valid instruction binary)
Format: H (000)
Flags Set: NONE
Operation: HLT

ADD
mne(opcode): **ADD** (0x00)
Format: R (001)
Flags Set: N | Z | V | C
Operation: R[Rd]= R[Rn] +R[Rm]

SUBtract
mne(opcode): **SUB** (0x01)
Format: R (001)
Flags Set: N | Z | V | C
Operation: R[Rd]= R[Rn] - R[Rm]

DIVide
mne(opcode): **DIV** (0x02)
Format: R (001)
Flags Set: N | Z | V | C
Operation: RIRd]= R[Rn] / R[Rm]

MODulus
mne(opcode): **MOD** (0x03)
Format: R (001)
Flags Set: N | Z | V | C
Operation: RIRd]= R[Rn] % R[Rm]

MULtiply
mne(opcode): **MUL** (0x04)
Format: R (001)
Flags Set: N | Z | V | C
Operation: R[Rd]= R[Rn] * R[Rm]

AND
mne(opcode): **AND** (0x05)
Format: R (001)
Flags Set: N | Z
Operation: R[Rd] = R[Rn] & R[Rm]

Inclusive OR
mne(opcode): **ORR** (0x06)
Format: R (001)
Flags Set: N | Z
Operation: R[Rd] = R[Rn] | R[Rm]

Exclusive OR
mne(opcode): **EOR** (0x07)
Format: R (001)
Flags Set: N | Z
Operation: R[Rd]=R[Rn] ^R[Rm]

Logical Shift Left
mne(opcode): **LSL** (0x08)
Format: R (001)
Flags Set: Z | V | C
Operation: R[Rd] = R[Rn] <<shamt

Logical Shift Right
mne(opcode): **LSR** (0x09)
Format: R (001)
Flags Set: Z | V | C
Operation: R[Rd] = R[Rn] >>> shamt

ADD Immediate
mne(opcode): **ADDI** (0x00)
Format: I (010)
Flags Set: N | Z | V | C
Operation: R[Rd] = R[Rn] + ALUImm

SUBtract Immediate
mne(opcode): **SUBI** (0x01)
Format: I (010)
Flags Set: N | Z | V | C
Operation: R[Rd] = R[Rn] - ALUImm

AND Immediate
mne(opcode): **ANDI** (0x02)
Format: I (010)
Flags Set: N | Z | V | C
Operation: R[Rd] = R[Rn] & ALUImm

Inclusive OR Immediate
mne(opcode): **ORRI** (0x03)
Format: I (010)
Flags Set: N | Z | V | C
Operation: R[Rd] = R[Rn] | ALUImm

Exclusive OR Immediate
mne(opcode): **EORI** (0x04)
Format: I (010)
Flags Set: N | Z | V | C
Operation: R[Rd] = R[Rn] ^ ALUImm

Load Register
mne(opcode): **LDUR** (0x00)
Format: D (010)
Flags Set: N | Z
Operation: R[Rt]=M[R[Rn] + DTAddr]

Load Byte Unscaled offset
mne(opcode): **LDURB** (0x01)
Format: D (010)
Flags Set: N | Z
Operation: R[Rt]={ 24'b0, M[R[Rn] + DTAddr][7:0] }

Store Register
mne(opcode): **STUR** (0x02)
Format: D (010)
Flags Set: NONE
Operation: M[R[Rn] + DTAddr] = R[Rt]

Store Byte Unscaled offset
mne(opcode): **STURB** (0x03)
Format: D (010)
Flags Set: NONE
Operation: M[R[Rn] + DTAddr][7:0] = R[Rt][7:0]

Branch to Register
mne(opcode): **BR** (0x0A)
Format: R (001)
Flags Set: NONE
Operation: PC=R[Rt]

Branch Relative Unconditional
mne(opcode): **B** (0x00)
Format: B (011)
Flags Set: NONE
Operation: PC = PC + BranchAddr

Branch with Link Relative
mne(opcode): **BL** (0x01)
Format: B (011)
Flags Set: NONE
Operation: R[30]=PC + 4; PC = PC + BranchAddr

Branch Equal
mne(opcode): **BEQ** (0x00)
Format: CB (100)
Flags Set: NONE
Operation: if(Z) PC + CondBranchAddr

Branch Not Equal
mne(opcode): **BNE** (0x01)
Format: CB (100)
Flags Set: NONE
Operation: if(!Z) PC + CondBranchAddr

Branch Less Than
mne(opcode): **BLT** (0x02)
Format: CB (100)
Flags Set: NONE
Operation: if(N!=V) PC + CondBranchAddr

Branch Less than or Equal
mne(opcode): **BLE** (0x03)
Format: CB (100)
Flags Set: NONE
Operation: if (Z || N!=V) PC + CondBranchAddr

Branch Greater Than
mne(opcode): **BGT** (0x04)
Format: CB (100)
Flags Set: NONE
Operation: if(!Z & N==V) PC + CondBranchAddr

Branch Greater Than or Equal
mne(opcode): **BGE** (0x05)
Format: CB (100)
Flags Set: NONE
Operation: if(N==V) PC + CondBranchAddr

Move wide with Keep
mne(opcode): **MOVK** (0x00)
Format: IW (111)
Flags Set: N | Z
Operation: R[Rd][SH*16+15:SH*16] = MOVImm

Move wide with Zero
mne(opcode): **MOVZ** (0x01)
Format: IW (111)
Flags Set: N | Z
Operation: R[Rd] = {MOVImm << (SH*16)}

Compare
mne(opcode): **CMP** (nan)
Format: PSUEDO
Flags Set: N | Z | V | C
Operation: XZR = R[Rn] - R[Rm]

Compare Immediate
mne(opcode): **CMPI** (nan)
Format: PSUEDO
Flags Set: N | Z | V | C
Operation: XZR = R[Rn] - MOVImm

MOVe
mne(opcode): **MOV** (nan)
Format: PSUEDO
Flags Set: N | Z | V | C
Operation: R[Rd] = R[Rm] - XZR

Call PPU
mne(opcode): **PPU**(0x0)
Format: ME (110)
Flags Set: NONE
Operation: NONE

FLUsh cache
mne(opcode): **FLU**(0x1)
Format: ME (110)
Flags Set: NONE
Operation: NONE

WAIt
mne(opcode): **WAI**(0x2)
Format: ME (110)
Flags Set: NONE
Operation: NONE

NOP
mne(opcode): **NOP**(0x0)
Format: ALIAS
Flags Set: NONE
Operation: NOP

CoMPare
mne(opcode): **CMP**
Format: ALIAS
Flags Set: N | Z | V | C
Operation: text replacement for SUB XZR, Xn, Xm

CoMPare Immediate
mne(opcode): **CMPI**
Format: ALIAS
Flags Set: N | Z | V | C
Operation: text replacement for SUBI XZR, Xm, IMM

MOVe
mne(opcode): **MOV**
Format: ALIAS
Flags Set: N | Z | V | C
Operation: text replacement for ORR Xd, XZR, Xn

LoaD Address
mne(opcode): **LDA**
Format: ALIAS
Flags Set: N | Z | V | C
Operation: text replacement for ORI Xd, XZR, IMM

Not register
mne(opcode): **Not**
Format: ALIAS
Flags Set: N | Z | V | C
Operation: text replacement for EORI Xd, Xn, -1

FOR loop
mne(opcode): **FOR**
Format: MACRO
Flags Set: NONE
Operation: Remplacement for comparisons and branches. Nesting is supported. Can compare register to value or register to register. Value to value not supported.

IF loop
mne(opcode): **IF**
Format: MACRO
Flags Set: NONE
Operation: Remplacement for comparisons and branches. Nesting is supported. Can compare register to value or register to register. Value to value not supported.

Assembler Directives:

Labels:

Formatted as “[Label Name]:”, it is used to ensure a location after assembly, as macros are expanded and sections are rearranged during preprocessing, leading to guarantees that instructions or data will be in the same locations as defined in the file. Labels are used in both .data and .text sections.

.text:

Section of the ASM that contains instructions.

.data:

Section of the ASM that contains data, using labels to mark their locations. Data in actual binary always goes after instructions.

.padding

Aligns the .text and .data sections with the cache to avoid accidental conflicts and data conflicts and avoid undefined behavior due to no inter-cache communication (ex., Modifying instruction space)

Memory/Cache System:

For our memory, we are using 64 KB of RAM, with byte addressing. With this sizing we will have 2^{16} addresses or 16 bits.

$$\frac{Ram\ Size}{Bytes\ Per\ Address} = Memory\ Addresses$$

The cache line size will be 8 words long to make for 3 offset bits for the words and 2 for offset bits for the bytes. We will have 32 cache lines for 5 index bits. The remaining 8 bits will be the tag. This makes for a total cache size of 1KB. It will be a direct map (due to the small size of the data) with a write-back cache policy. We hope to store the majority of the current use instructions in the cache while still ensuring misses do occur occasionally for simulation purposes. A stretch goal will be to implement a second level to the cache to learn the process.

Our system will offer a Harvard architecture for our cache meaning we will have a data and instruction cache. Each of these cache's will follow the previous written guidelines and will be 1 KB in size. Since the cache will consist of 32 lines of 8 words this allows us to store 256 instructions. This will be ample size to load most small programs but will still be small enough to fill and ensure proper cache line evictions.

Cache Address					
Dirty bit	Valid bit	Tag	Index	Word Offset	Byte Offset
1 bit	1 bit	6 bits	5 bits	3 bits	2 bits

Benchmark Plan:

ISA Sim Benchmarks:

To test our system we will need to implement two key benchmarks. The first benchmark will be a matrix multiplication. Likely we will start with simple 2x2 or 3x3 matrix multiplication operations and then expand outwards to larger sizes once we can determine the ISA simulator functions as expected (example a 50x50 matrix). Larger matrix multiplication operations will ensure the cache gets completely filled and will need to handle misses. For each matrix multiplication, the output will be saved to be compared to the true answer for verification.

The second benchmark will be performing an exchange sort algorithm. Likely we will push through a set of instructions to sort a large set of numbers. Once the ISA simulation is finished it will be simple to determine if the ISA performed correctly by examining the result ordering.

Outside of correctness, we can examine performance metrics for our system. Important metrics such as the hit rate, CPI, end clock count, and general execution time.

Graphical Benchmark:

For graphics, we will load a sequence of bytes into the area defined in **Appendix C** to display an image via PPU. The benchmark will be considered successful if the image is displayed accurately based on the image hoped to be displayed.

ASI Implementation:

The backend (the actual simulation of the system) will be coded in C++. The system will be divided into multiple self-contained modules that will function independently from each other to allow for a more controlled development. The main components of the build will be the CPU, PPU, and RAM. Making it components will allow for debugging to occur more effectively.

UI implementation:

To implement the UI our group will code a GUI in C++ using Qt as the main library to implement the UI. Qt provides all of the necessary tools along with quality documentation to make the coding easier. The UI will be integrated within the simulation software to maximize the ease of calling the UI and passing data to and from the UI. The UI and simulation will run in two separate threads that will use signals and slots for communication. A multithreaded operation will increase the reactivity of the UI while maintaining simulation speeds. Qthreads where used for threading, a worker class was created for handling simulation events. Events include, but are not limited to, actions such as single stepping and loading a program into memory.

The GUI contains the following information:

- Data Tables
    - Pipeline
    - Register
    - Cache Memory
- Cycle Operations
    - Single cycle step
    - Run X cycles
    - Run to completion
- Program Configuration
    - Load program
    - Reset program
    - Cache/pipeline disable toggle
- Run-time variables
    - PC
    - Cycle count
    - Halted T/F

Management (Old):

A list of tasks will be made in Google Sheets to keep track of progress in the project. We will communicate via email and discuss our project weekly either in person or remotely.

We will work together to finalize the API design and ISA, together with creating a pipeline between the GUI and the main backend component. Once it is established, Pablo will start working on the Memory/Cache while James will work on a draft for the UI. Once done with memory, James and Pablo will test the program and attempt an initial state dump and load. Once the UI is near finished, James will help with the development and debugging of the pipeline. Each part of the project will have to pass a GUI test to make sure that each part is compliant with the API established earlier. Pablo and James will work together on the final benchmark and report, with James focusing on the UI description and Pablo on the simulator and benchmarks.

Appendix A Register Description

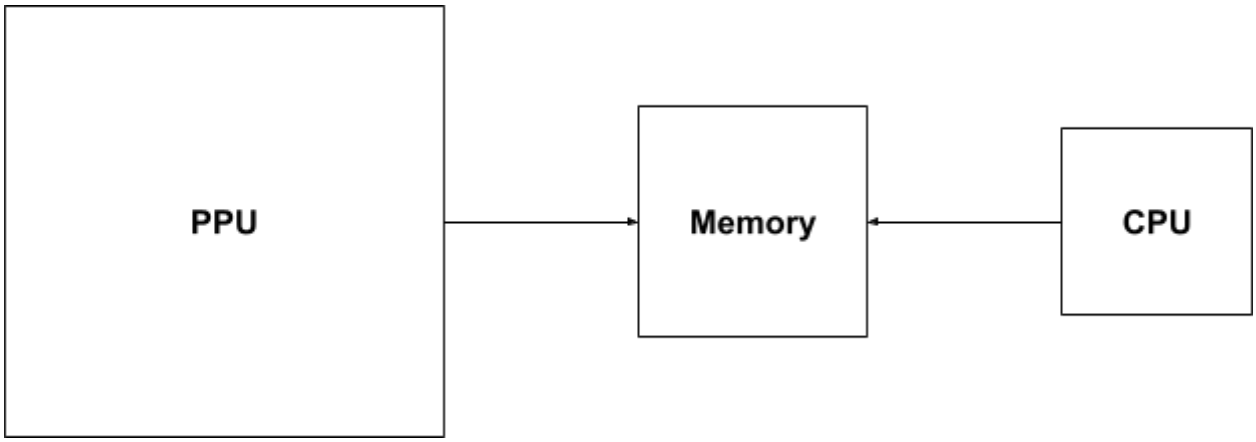
NAME	NUMBER	USE
X0 - X12	0 - 12	General Purpose
X13 (SP)	13	Stack Pointer
X14 (PC)	14	Program Counter
XZR	15	Zero Register
FLAGS	NA	Negative | oVerflow | Zero | Carry

Appendix B: Opcode Table

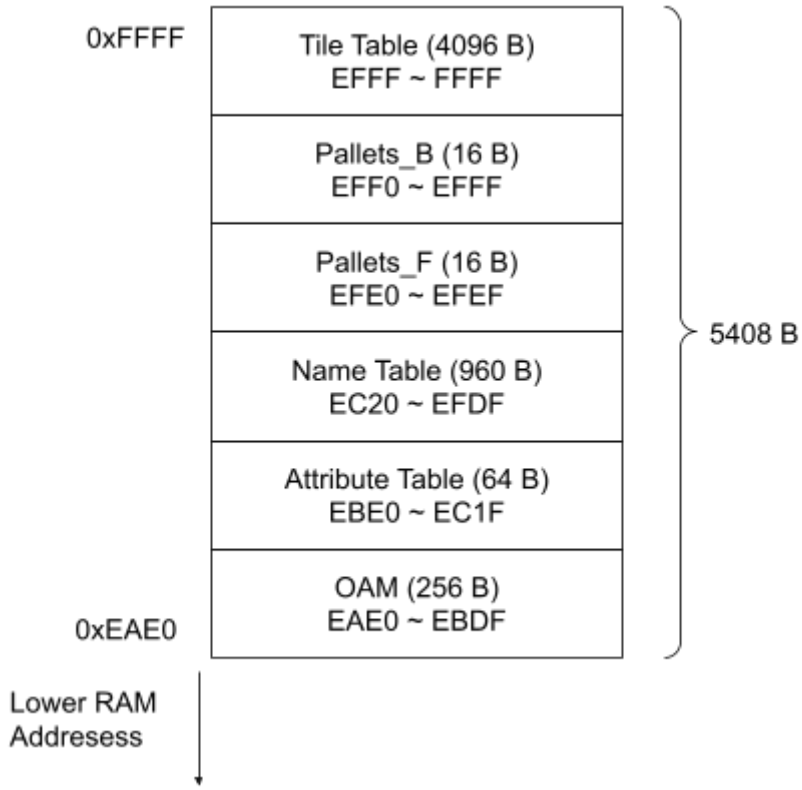
Name	Mnemonic	Sets Flags	Format	Opcode (Hex)	Operation
HLT	HLT	NONE	H (000)	0x00	HLT
ADD	ADD	N | Z | V | C	R (001)	0x00	R[Rd]= R[Rn] +R[Rm]
SUBtract	SUB	N | Z | V | C	R (001)	0x01	R[Rd]= R[Rn] - R[Rm]
Signed DIVide	DIV	N | Z | V | C	R (001)	0x02	RIRd]= R[Rn] / R[Rm]
MODulus	MOD	N | Z | V | C	R (001)	0x03	RIRd]= R[Rn] % R[Rm]
MULtiply	MUL	N | Z | V | C	R (001)	0x04	R[Rd]= R[Rn] * R[Rm]
AND	AND	N | Z	R (001)	0x05	R[Rd] = R[Rn] & R[Rm]
Inclusive OR	ORR	N | Z	R (001)	0x06	R[Rd] = R[Rn] | R[Rm]
Exclusive OR	EOR	N | Z	R (001)	0x07	R[Rd]=R[Rn] ^R[Rm]
Logical Shift Left	LSL	Z | V | C	R (001)	0x08	R[Rd] = R[Rn] <<shamt
Logical Shift Right	LSR	Z | V | C	R (001)	0x09	R[Rd] = R[Rn] >>> shamt
Branch to Register	BR	NONE	R (001)	0x1A	PC=R[Rt]
ADD Immediate	ADDI	N | Z | V | C	I (010)	0x00	R[Rd] = R[Rn] + ALUImm
SUBtract Immediate	SUBI	N | Z | V | C	I (010)	0x01	R[Rd] = R[Rn] - ALUImm
AND Immediate	ANDI	N | Z | V | C	I (010)	0x02	R[Rd] = R[Rn] & ALUImm
Inclusive OR Immediate	ORRI	N | Z | V | C	I (010)	0x03	R[Rd] = R[Rn] | ALUImm
Exclusive OR Immediate	EORI	N | Z | V | C	I (010)	0x04	R[Rd] = R[Rn] ^ ALUImm
Load Register	LDUR	NONE	D (010)	0x05	R[Rt]=M[R[Rn] + DTAddr]
Load Byte Unscaled offset	LDURB	NONE	D (010)	0x06	R[Rt]={ 24'b0, M[R[Rn] + DTAddr][7:0] }
Store Register	STUR	NONE	D (010)	0x10	M[R[Rn] + DTAddr] = R[Rt]
Store Byte Unscaled offset	STURB	NONE	D (010)	0x11	M[R[Rn] + DTAddr][7:0] = R[Rt][7:0]

Branch	B	NONE	B (100)	0x00	PC = PC + BranchAddr
Branch Equal	BEQ	NONE	B (100)	0x01	if(Z) PC + CondBranchAddr
Branch Not Equal	BNE	NONE	B (100)	0x03	if(!Z) PC + CondBranchAddr
Branch Less Than Signed	BLT	NONE	B (100)	0x04	if(N!=V) PC + CondBranchAddr
Branch Less than or Equal Signed	BLE	NONE	B (100)	0x05	if (Z || N!=V) PC + CondBranchAddr
Branch Greater Than Signed	BGT	NONE	B (100)	0x06	if(!Z & N==V) PC + CondBranchAddr
Branch Greater Than or Equal Signed	BGE	NONE	B (100)	0x07	if(N==V) PC + CondBranchAddr
Branch Less Than	BLO	NONE	B (100)	0x08	if(C) PC + CondBranchAddr
Branch Less Than or Equal	BLS	NONE	B (100)	0x09	if(C || Z) PC + CondBranchAddr
Branch Greater Than	BHI	NONE	B (100)	0x0A	if(!C & !Z) PC + CondBranchAddr
Branch Greater Than or Equal	BHS	NONE	B (100)	0x0B	if(!C) PC + CondBranchAddr
MOVe wide with Keep	MOVK	NONE	IM (101)	0x00	R[Rd][SH*16+15:SH*16] = MOVImm
MOVe wide with Zero	MOVZ	NONE	IM (101)	0x01	R[Rd] = {MOVImm << (SH*16)}
call PPU	PPU	NONE	ME (110)	0x0	None
FLUsh cache	FLU	NONE	ME (110)	0x1	None
WAIt	WAI	NONE	ME (110)	0x2	None
No Operation	NOP	NONE	NOP (111)	0x0	NOP
CoMPare	CMP	N | Z | V | C	ALIAS	NONE	text replacement for SUB XZR, Xn, Xm
CoMPare Immediate	CMPI	N | Z | V | C	ALIAS	NONE	text replacement for SUBI XZR, Xm, IMM
MOVe	MOV	N | Z | V | C	ALIAS	NONE	text replacement for ORR Xd, XZR, Xn
LoaD Address	LDA	N | Z | V | C	ALIAS	NONE	text replacement for ORI Xd, XZR, IMM
Not Register	NOT	N | Z | V | C	ALIAS	NONE	text replacement for EORI Xd, Xn, -1
FOR loop	.FOR	NONE	MACRO	NONE	Remplacement for comparisons and branches
IF loop	.IF	NONE	MACRO	NONE	Remplacement for comparisons and branches

Appendix C PPU draft functioning



Data Stack for PPU



PPU Functioning

Inspired by NES PPU (https://www.nesdev.org/wiki/PPU **)**

The PPU uses 6 distinct data groups to perform its operations:

- Tile Table
- Pallets B

- Pallets F
- Name Table
- Attribute Table
- OAM (Object Attribute Memory)

Pallets B and Pallets F determine the foreground and background pallets. Each group is 4 pallets long, 4 bytes long, each pointing to a different color determined by an internal table.



(photo from Wikipedia
https://en.wikipedia.org/wiki/List_of_video_game_console_palettes)

The Tiletable (AKA Pattern Table) contains “tiles”, single units of 8x8 pixes each made up of 16 bytes, allowing each pixel to be labeled with a value from zero to 3. This allows the tile to display 4 distinct colors, determined by the pallet for that tile.

Given a row index of n and column index of m (going left to right, up to down) the pixel would be determined by the mth bit (most significant bit) of the n byte of the tile and the mth bit of the n+16 byte of the tile.

```
def get_pixel_code(tile, n, m):
    a = tile[n] & 1 << (7-m) ? 0b10 : 0
    b = tile[n] & 1 << (7-m) ? 0b1 : 0
    return a+b
```

The tiles can be used for 2 purposes, as a sprite or a background object.

The Nametable is the actual display of the screen (256x224), with each byte representing an 8x8 space occupied by a tile. The byte references a tile within the Tiletable to be rendered at that specific location. The colors of the tile are determined by the **Attribute Table**. The attribute table is 1 byte for 2x2 tiles groups (starting left up) (two bits per tile), allowing each tile to reference a different pallet in the pallet_b table. The order for giving a pallet goes as follows.

0	2
1	3

The OAM determines the sprites on the screen. Each sprite is 4 bytes long.

- Y-coordinate (pixels)
- The tile used
- Descriptor: HVXXXXPP
    - Horizontal Flip (the sprite is flipped horizontally)
    - Vertical Flip (the sprite is flipped vertically)
    - Pallet
- X-coordinate (pixels)

Each of these determines the output of the sprite. It should be noted that for 0 of any pallet, the color is ignored allowing for the sprite to mesh into the background.

Note: This an EXTREMELY simplified version of the original PPU, lacking many of the features of the original.

Simulator Differences

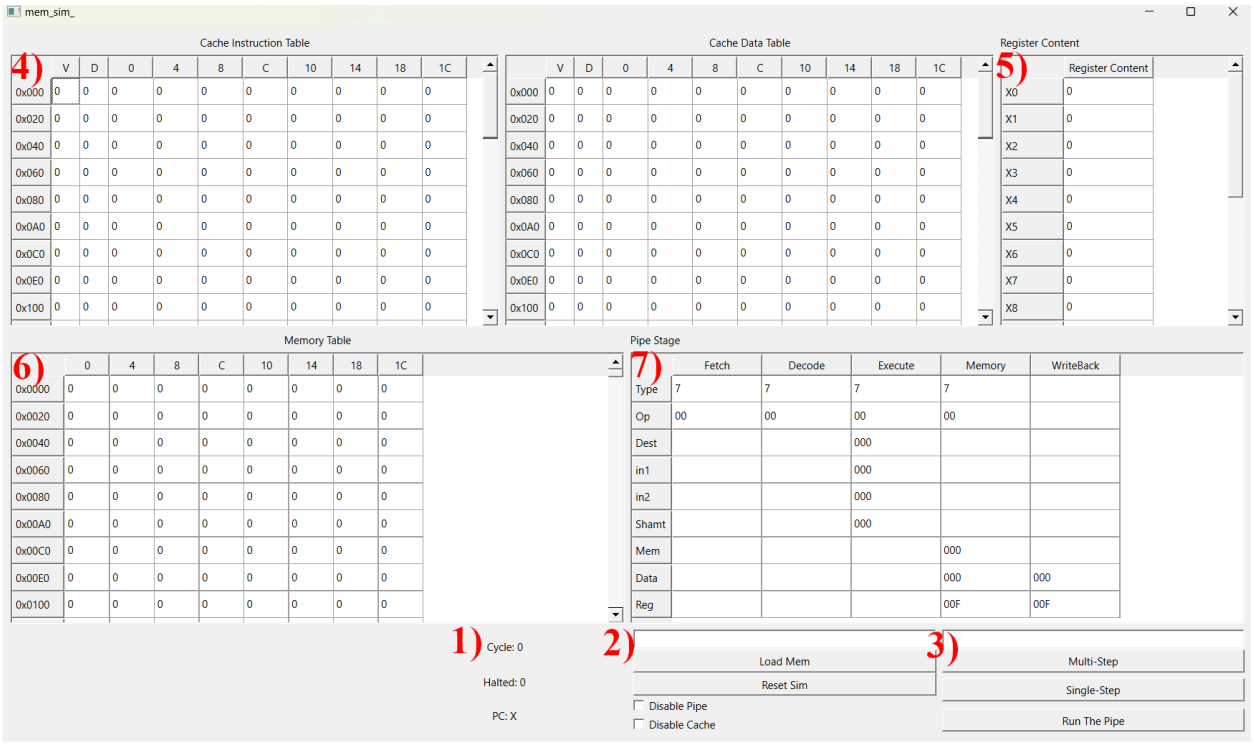
The current ISA description above is updated with the changes we made. Here is a list of changes we made to the original draft proposal.

- We removed registers that were in our original design, such as link registers and a I/O register. The link register was not used at any point and was simply reserving space for an unused operation, making it pointless. The I/O register was also deemed redundant as the data from the input didn't necessarily need to be in one specific register, so we instead changed the input instruction to be able to specify a register as well. These two registers are instead now General Purpose registers. Previously general purpose registers were X0-X10, now X0-X12.
- After recommendations by our instructor, we realized that having unsigned operations was unnecessary.
- Some instructions were removed due to being unnecessary.
- "Meta" instructions were added that allowed for more control over the PPU and when the main memory was updated from cache.
- More aliases and pseudo instructions were added. Macros and compiler directive were also added to the specifications as they were added to the assembler.
- Originally the base instruction format type field was 4 bits and the opcode was five. This was Adjusted to allow for more generalizing for certain instructions.
- Program now is multithreaded, one thread for GUI and one for the simulator.

Simulator Description

User Manual:

Below is an image of our simulators UI with numbered descriptors next to important sections. Descriptions will be found below.



1)

Section one provides some runtime data. This data includes the current clock cycle, if the system has been halted and the Program Counter.

2)

Section two is the user setup panel. This is where you will be able to load pre-assembled programs into the simulation. A folder named “load” will be found in the software package which is where all programs will need to be located. To load a program simply type the name of the file and click “Load Mem”. This will load the program and will show data in the Memory table once loaded.

At any point during the simulation you may select “Reset Sim” which will clear all data loaded in the Memory Table, Cache Table, Pipeline and any other unseen variables used by the simulation. Use the reset button before loading in a new program.

“Disable Pipe” and “Disable Cache” allow for simple toggling of various simulator modes. It is recommended to configure your simulator mode before you begin to run the program.

3)

Section three is the runtime panel. It allows the user to single step through the program using the "Single-Step" button. To step an arbitrary number of steps simply type in a decimal number in the text bar and push the "Multi-Step" button. This can be done repeatedly. To run the loaded program to the end select "Run The Pipe". This will execute all loaded instructions until a HLT instruction is found. Once done the reset button can be used and will allow you to load a new program.

The UI data sections will update immediately after a single-step, multi-step, or run-the-pipe action has completed.

4)

Section four has two tables, one for the Cache Instruction Table and one for the Cache Data Table. The Cache follows a Harvard type Architecture. The Cache Instruction Table will display all instruction lines loaded from the memory. The Cache Data Table will show all data during run time that has been saved to the cache from either the registers or the memory.

5)

Section five shows the data contained in the system's 16 registers and the Reg Buffer. For specifics of what each register corresponds to please refer to **Appendix A**.

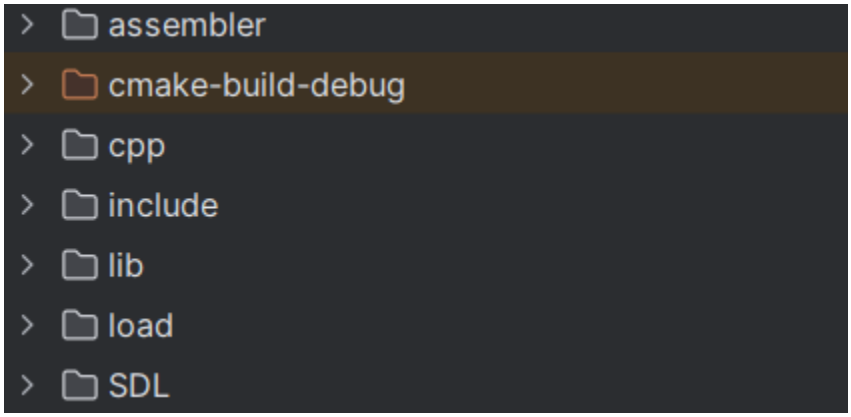
6)

Section six refers to the systems Memor Table. This table will hold program data upon initial loading and any data that the program specifies to save to the memory.

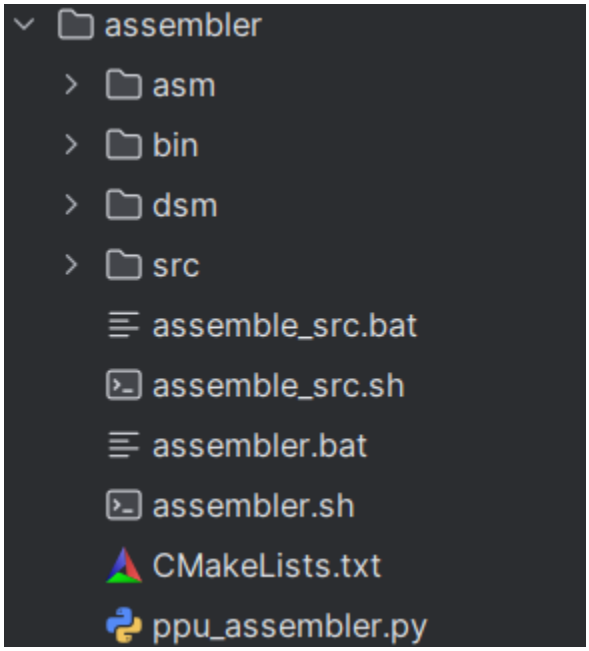
7)

Section seven shows Pipe Stage data. Any data currently stored inside the pipe will be displayed here. Fetch, decode, execute, and memory stage's each show the instruction type and op code loaded in that stage of the pipeline. Execute stage shows a more descriptive breakdown of the contents of the instruction. Memory and writeback shows data relating to store operations.

Code/Folder Structure:



assembler)

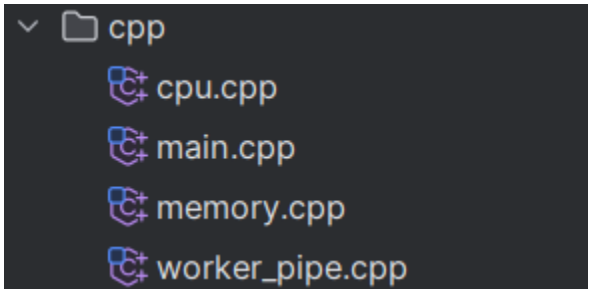


asm: Generated assembly language files with macros resolved, human-readable
bin: Generated machine language files, used to load the simulator
dsm: Generated detailed compiler notes
src: User assembler code to be compiled
ppu_assembler.py: Assembler script, written in Python

cmaker-build-debug)

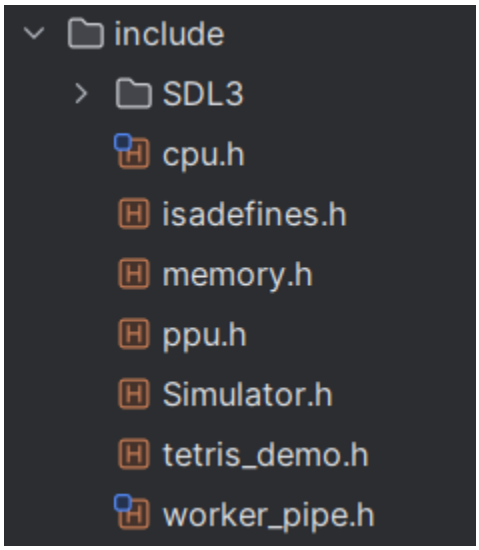
Used for Code compilation in the CLION IDE

cpp)



cpu: Pipeline source code, UI code
main: Source code, initialized simulation and then executes the UI
memory: Source code for the memory/cache system
worker_pipe.cpp: Source code for the UI multithreading and simulation tools

include)



sdl) UI package used for tetris/PPU
cpu: cpu.cpp declarations
isadefines: Macro defines
memory: memory.cpp declarations
ppu: Declarations and definitions for PPU class
Simulator: No longer used
tetris_demo: No longer used
worker_pipe: Declarations for worker_pipe.cpp

lib)

SDL Binary files

load)

Pre-compiled programs for the user to load

SDL)

SDL Binary files

Engineering Methods

Problem Identification:

Like any good engineer, we needed to identify our problem and create a solution. To design the UI for our system, we first had to examine the requirements. We needed a UI that was relatively performant and could simply be integrated into our C++ script. Qt fits all of our criteria and has a significant amount of resources available for learning. Because of this, we were able to multithread a program to make a highly responsive UI.

For our storage and pipeline system, we researched how real ARM architectures function. Reading documentation such as ARM greencards, manuscripts, and online articles we were able to grasp a solid foundation for how we need to implement our system. For any unforeseen problems during the development phase, we had office hours available to clarify implementation questions. For example, Prof Weems was a valuable resource when fixing odd bugs in our pipeline.

Development:

Using resources available such as Qt documentation and example code found online we were able to formulate a UI that fitted our specific needs. Integrating these tools into our pre-existing code allowed us to have an easy line of communication between the simulation and the UI. Using tools like AI and pre-written code we were able to figure out the Qt syntax for designing our UI. Using separate smaller scripts to test out specific features for simpler tasks like making a button to more complicated tasks such as threading was a vital step in the process. Once the syntax was learned we were able to easily port over features like Qt threads into our simulation. An example here is when we found out that the UI has to run in main, this caused some slight redesign in our simulation script. Moving our simulation code into a separate thread to allow for the UI to run in main. Then using signals and slots seamless communication between the two threads was formulated.

Numerous coding methods were used in the development phase that helped contribute to a clean-looking coding project. Macros were used to allow a more readable script that was simpler to update. Object-oriented programming offered in C++ allowed us to separate our code into smaller core chunks (CPU, PPU, CACHE, MEMORY, BUS MEMORY HANDLER, GUI, Threading) which communicate via a standard API. Memory handlers and Cache share an API to reduce the amount of code required to communicate between the modules. This method allowed us to split the coding efforts and work in parallel without worrying you're working on an outdated script. Tools like Github were used to share code, while Discord was the primary source of wireless communication.

Debugging:

All of our debugging was done by hand. Tools like the UI and IDE debuggers allowed us to slow down our code and pinpoint specific coding errors that led to simulation errors. An IDE debugger allowed us to single step our program and view variable data to allow us to compare expected results to perceived results. Slowing down the runtime allowed us to discover the exact moments that lead to unexpected results.

The UI data tables allowed us to view all of the simulation storage including the registers, cache, memory, and pipeline steps. Similar to the Debugger, single stepping through our simulation allowed us to figure out what cycle the programs started to malfunction on. Looking at data like the PC and register table allowed us to discover small errors such as control bits not setting causing branching to behave not as expected.

Work Efforts

Both James and Pablo kept close communication throughout the whole development process to ensure a fair split of the work. James was able to focus on the GUI, multithreading, and demo day plans. Pablo was able to focus on some of the core simulation features finishing most by spring break. Both James and Pablo contributed to the code design, debugging, and project submission requirements. Below is a breakdown of the main tasks in the project.

Task	Pablo	James
ISA definition	**X**	**X**
GUI		**X**
Testing	**X**	**X**
Assembler	**X**	
Simulation	**X**	**X**
PPU	**X**	
Benchmarks	**X**	**X**
Debugging	**X**	**X**
Demo day plans		**X**
Report writing	**X**	**X**

ISA definition:

Both James and Pablo planned the structure of ISA. James offered the original idea of adapting the simplified ARM instructions, LegV8, while adding suggestions for unnecessary instructions for the implementation. Pablo simplified the OP code section, increasing the size of the opcodes to create byte alignment to allow for simpler mask implementation in the ISA simulator (0xFF). James improved upon the opcode formats to also align the instructions information with bytes.

GUI:

The first step in the GUI was learning how QT works. This consisted of learning QThreads, signals and slots, Qwindows, formatting the window, the various QT objects such as Qlabels and other smaller tasks such as populating tables and updating the GUI. The GUI consisted of multithreading the program. One Thread runs the Simulation and the other runs the GUI. The two threads communicate

through QT signals and slots. James had never made a GUI before so this was a great learning opportunity to learn the QT framework.

Testing:

Pablo wrote the majority of the Benchmarks, he then ran them and collected the data. James then graphed all of the data, organized it in a table and did some analysis. Both Pablo and James assisted in writing the testing section in the report.

Assembler:

Pablo planned and developed the assembler in python before the ISA was developed to allow for testing at all levels of the implementation. Pablo added various non-defined instructions to serve as macros, assembler directive, or substitutions to allow for more comfortable developer experience. He coded the assembler to allow for a data section so information could be preloaded as part of the binary file and allowed for binary, hexadecimal, and decimal numbers to be used. He also implemented labels to avoid jump misalignment due to precompilation adding or removing lines.

Simulation:

The simulation was a team effort, with Pablo handling most of the PPU and memory components, and some of the CPU API, while James focused on creating a robust architecture and framework. James and Pablo shared the work of the pipeline and non-pipeline implementations, James focused on debugging logic errors, Pablo focused on cycle consistency and memory communication.

PPU:

Pablo developed the PPU as a simplified implementation of the original NES PPU. Most of the work was implementing the PPU as defined in the original implementation of NES, modified to directly read from RAM rather than go through an interface of communication like the original. SDL integration to display was completed after integration and allowed the results to be displayed in a seamless manner.

Benchmarks:

Benchmarks were a joint effort by Pablo and James. James worked on most of the logic of the code and Pablo, who was more familiar with the assembler and the assembly language used, coded them as efficiently as he possibly could. Both performed the actual execution of the code, in an effort to make sure the results were architecture and operating system independent, but also to debug possible issues caused by simulator failures rather than benchmark implementation failures.

Debugging:

Debugging had two main concentrations. As you develop your code you need to ensure your code functions as you add new additions to it such as a table in the GUI or a new instruction type type to the ISA. Pablo focused on debugging the simulation during development and James focused on debugging the GUI during development. The second main phase of debugging is after you have a working coding project. Both Pablo and James would run each other's code to look for edge case's and any way we could break the simulation. It was especially useful to get a different perspective on running our code, this was helpful for finding edge cases in both the ISA and the GUI. Both Pablo and James found multiple odd edge case bugs that broke the simulation but were able to be properly patched before the demo.

Demo day plans:

Demo day plans were simple. It mostly consisted of planning the best series of programs to run during the demo. The programs had to be chosen wisely to ensure we showcase all of our simulators features in the smallest amount of time possible. The first demo consisted of planning a small list of commands to run inside of our cache/memory system terminal. The second demo consisted of showing off our GUI and loading a program that showed off all instructions types to ensure we can branch, load/store, and add. Also ensuring our pipeline was properly being displayed. The final demo consisted of us showing off our bubblesort, matrix mult, and tetris demo on each of the four different modes.

Report writing:

Both James and Pablo equally contributed to writing the report. For the project proposal we had both written large chunks of our report such as Pablo writing the ISA instruction type and register section and James wrote the memory/cache specifications and General description section. For the Final report again both James and Pablo contributed to writing the report. James wrote for example the Simulation description section and Pablo modified changes we made to the ISA in the report.

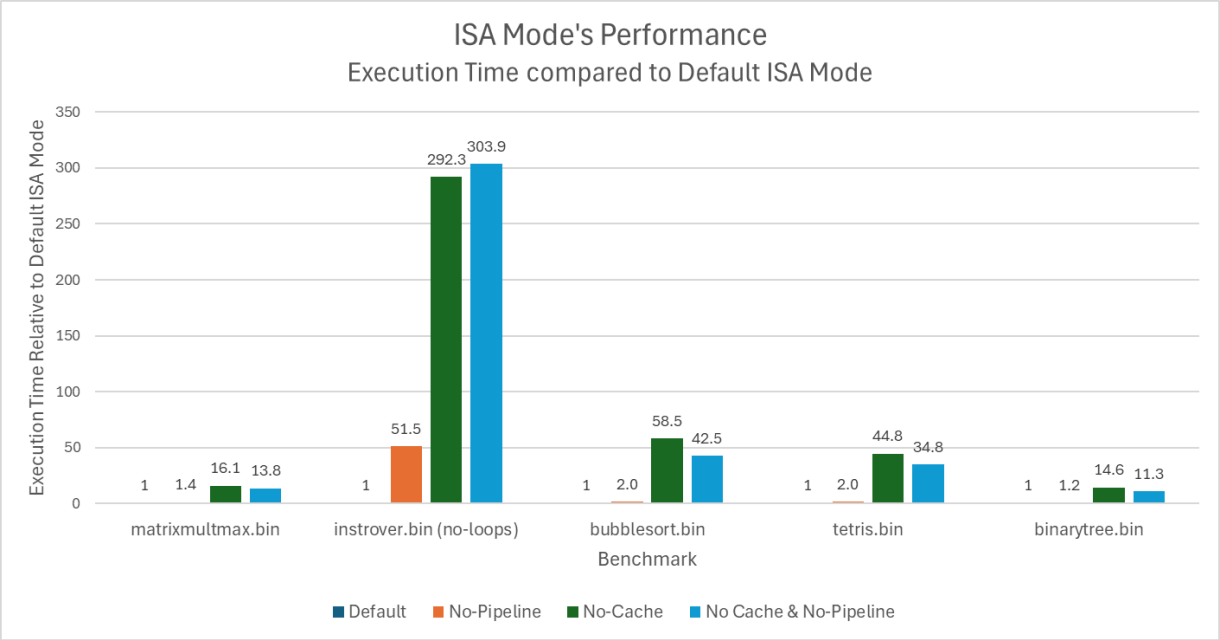
Performance Metrics

Bench Mark Program	ISA Mode and Program Cycle Count				Cache Hit Rates			
					Data Cache		Instruction Cache	
	Default	No-Pipeline	No-Cache	No Cache & No-Pipeline	Hit	Miss	Hit	Miss
matrixmultmax.bin	14,217,859	19,304,085	228,548,402	196,899,828	143360	109140	14217354	5
instrover.bin (no-loops)	103	5,305	30,102	31,304	0	0	716	38
bubblesort.bin	17,473,500	34,448,710	1,022,182,102	743,261,408	1943160	54840	17473197	3
tetris.bin	103,480	204,474	4,633,946	3,603,944	2333	323	102573	8
binarytree.bin	1,998	2,459	29,154	22,640	43	5	1093	7

Bench Mark Program Description

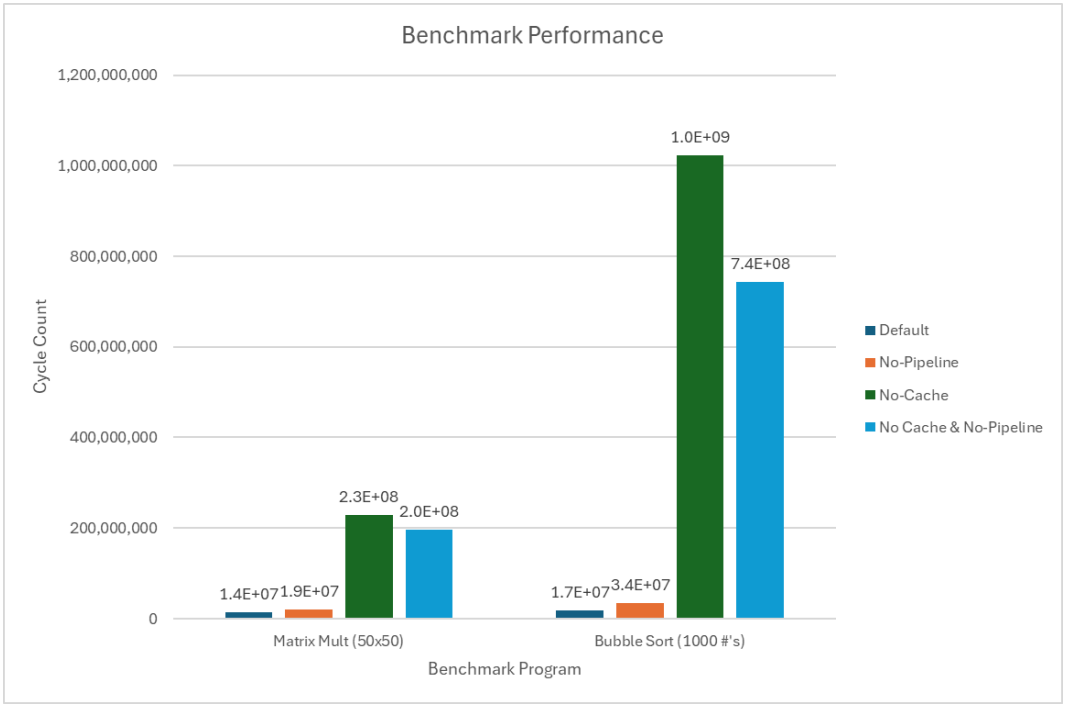
- Matrixmultmax.bin: a 50x50 matrix mult
- instrover.bin (no-loops): Simple 300 immediate adds to fill the cache and ensure proper evictions
- Bubblesort.bin: 1000 number bubble sort
- Tetris.bin: Runs tetris on the PPU unit
- Binarytree.bin: Navigation of a small a binary tree

A surprising result we found was that pipeless and cacheless were faster than just cacheless. instrover.bin (which contains no loops) confirmed our suspicions. The “slower” performance was caused by a lack of effective branch predictor, leading to unneeded instruction fetches and pipe clears, and with each memory load costing 100 cycles, it caused a significant dip in performance. With a cache, pipeline easily outperforms no-pipeline by a large margin in all benchmarks.
In terms of hits and misses, we found that data fetching caused the most amount of misses and instruction cache benefited from loops, compared to cacheless, where loops were a hindrance to pipelines. A better architecture would likely decrease the size of the Instruction cache and increase the size of the data cache to decrease the number of misses, therefore improving performance.

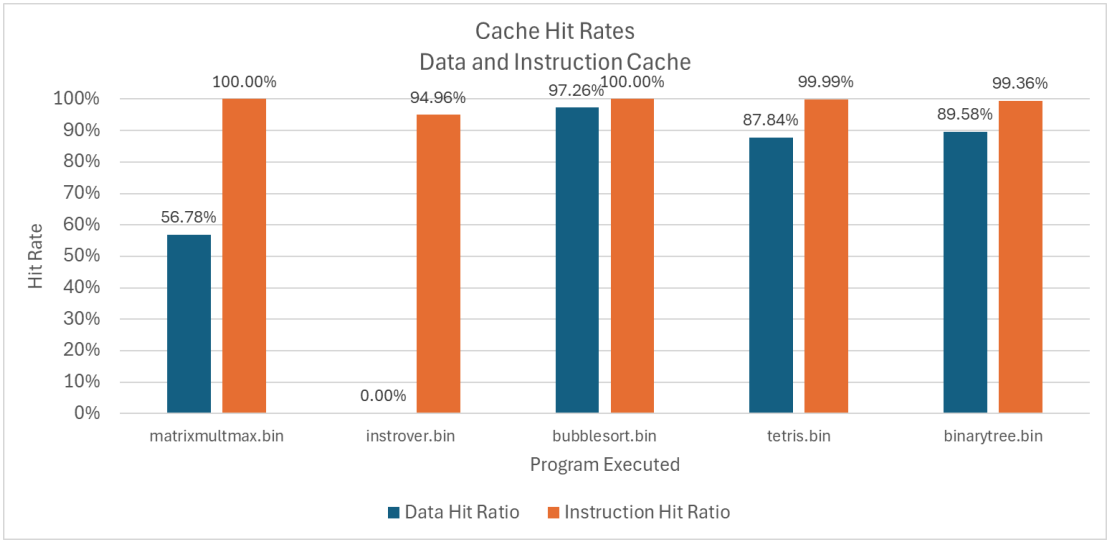


Graph 1: Bars define ISA mode, from left to right; Default, No-Pipeline, No-Cache, No Cache & No-Pipeline

Displays the time it takes to run each benchmark in each of the four modes in multiples of the Default mode run time for that benchmark. Example; in the matrixmultmax.bin, it takes 1.4 times longer to run in No-Pipeline, 16.1 times longer for No-Cache, and 13.8 times longer to run in No Cache & No-Pipeline mode than the Default mode for the matrixmultmax.bin program.



Graph 2: 1000 number bubble sort cycle count vs 50x50 matrix multiply cycle count



Graph 3: Cache hit rates for each benchmark. Hit rates are for the Data Cache and Instruction Cache separately.

Note: instrover.bin does not access the Data Cache resulting in an undefined hit rate.

Project Summary

This had been a significant learning experience for both of us. We learned both technical experiences relating to coding and how ISA functions in a more in-depth look. Both of us have had some degree of architecture experience before this course. Pablo in his free time often coded and explored architectures such as programming his own custom NES emulator. James had taken ECE331 and learned about ARM LEGv8 along with ECE558/559 (VLSI). This course has further developed our understanding of the Computer Architecture field.

We learned a good bit about new programming frameworks such as Qt. This is a useful skill that will likely translate into other applications. Similarly, we used SDL for the PPU to allow us to visually see the Tetris game. Something that was not expected was for the simulator to be too slow to completely emulate Tetris. While key bindings did function, the game was far too laggy to play. We learned how much of a bottleneck memory was. While we both understood that memory is slow and has not scaled much in the past decade compared to transistor technology, we did not expect to see the delays we did in our simple simulation from memory. This was a learning point for why modern systems need to have so much planning for memory access. When we implemented memory-only mode it was astounding how much slower the system was. Branch instructions also have a bigger point of delay than expected but it makes sense. We need to empty the pipeline whenever a branch is taken.

Another key learning point is the level of complication in keeping track of memory/cache delays when load/store instructions are taken. One might assume that simply waiting a few cycles is sufficient. However, the process involves significantly more complexity than merely introducing a delay. A non-trivial amount of control logic is required to keep errors from propagating throughout the system. Additionally, these delays can cause the whole system to bog down.

There were more nuanced things we learned as well. Many libraries have complex requirements for compilation. Cross platform development is not only complex but unnecessarily hard to find documentation for. Certain frameworks such as QT do not work with C++ templates which caused us to rewrite certain classes. We also further develop our project management skills. This included partitioning the work for a large project, communicating code updates and keeping on top of uploading the latest versions of our code. Other factors too such as cooperating with each other's very busy schedules and working with each other. Example James had a lot of SDP work to do so Pablo was able to help out a bit more on the week's James had his presentations. Pablo went on vacation for a couple of weeks and James was able to help out in the time of his absence.

It is always pleasant to get team coordinating experience where multiple members need to work in congruence with one another. This included the fact that different operating systems were used, requiring the need for a cross-platform

compilation method. Ensuring each member accomplished their assigned tasks can be stressful for all members but luckily both of us worked great with one another.

Overall this was a great learning experience for both of us, we learned a lot about computer architecture systems and all the complexities that are involved under the hood.